

Context: Act as an Automation Architect to design the framework for RestAssured API Automation with TestNG

Instructions: [MANDATORY] Ask me 7 Multiple choice questions one by one [MANDATORY] Once after answering the first question, proceed with next question [MANDATORY] Generate Framework structure first after completing all the questions [STRICT] Proceed Step by step for framework implementation [STRICT] Upon approval proceed with the next step
Alright Srinivasan, let's begin step by step. I'll ask you **7 multiple-choice questions one by one** to tailor the RestAssured + TestNG framework design. After you answer all, I'll generate the framework structure and proceed with implementation steps.

Question 1: Project Build Tool

Which build tool do you prefer for managing dependencies and running the automation framework?

A) **Maven** B) **Gradle** C) **Ant** D) **No preference, use default**

👉 Please select one option.

A

Perfect, Srinivasan — you've chosen **Maven** as the build tool. That gives us a solid foundation for dependency management and CI/CD integration.

Question 2: Test Execution & Reporting

Which reporting mechanism would you prefer to integrate with TestNG for clear test execution results?

A) **TestNG default reports** B) **Extent Reports** (rich HTML reports with screenshots/logs) C) **Allure Reports** (interactive, modern reporting) D) **Minimal reporting, focus on console logs**

👉 Please select one option.

C

Great choice, Srinivasan — you've selected **Allure Reports (C)**. That will give us modern, interactive reporting with detailed insights into API test runs.

Question 3: Test Data Management

How would you prefer to manage test data for your API automation framework?

A) **Hardcoded within test classes** (quick but less flexible) B) **External files (JSON, Excel, CSV, YAML)** C) **Database-driven test data** D) **Environment-specific property files**

👉 Please select one option.

B

Excellent, Srinivasan — you've chosen **External files (JSON, Excel, CSV, YAML)** for test data management. That gives us flexibility and scalability for different environments.

Question 4: API Environment Handling

How would you like to manage multiple API environments (e.g., Dev, QA, Prod)?

A) **TestNG XML parameters** B) **Property files per environment** C) **Environment variables (system-level)** D) **Centralized Config Manager class**

👉 Please select one option.

B

Excellent, Srinivasan — you've selected **Property files per environment (B)**. That's a clean way to handle Dev/QA/Prod configurations without overcomplicating the setup.

Question 5: API Request/Response Validation

How would you prefer to validate API responses in your framework?

A) **Status codes only** B) **Status codes + Response body fields** C) **Schema validation (JSON schema)** D) **Combination of all (status, body, schema)**

👉 Please select one option.

D

Perfect, Srinivasan — you've selected **D (Combination of all: status, body, schema)**. That ensures robust validation and makes the framework enterprise-grade.

Question 6: Test Execution Strategy

How would you prefer to organize and execute your API tests?

A) **Single suite with all tests** B) **Modular suites per API service/domain** C) **Tag-based execution (Smoke, Regression, Sanity)** D) **Hybrid approach (modular + tags)**

👉 Please select one option.

C

Excellent, Srinivasan — you've selected **C (Tag-based execution: Smoke, Regression, Sanity)**. That will give us flexibility to run targeted suites depending on the testing need.

Question 7: CI/CD Integration

Which CI/CD tool would you prefer to integrate this RestAssured + TestNG framework with?

A) **Jenkins** B) **GitHub Actions** C) **GitLab CI/CD** D) **Azure DevOps**

👉 Please select one option.

C

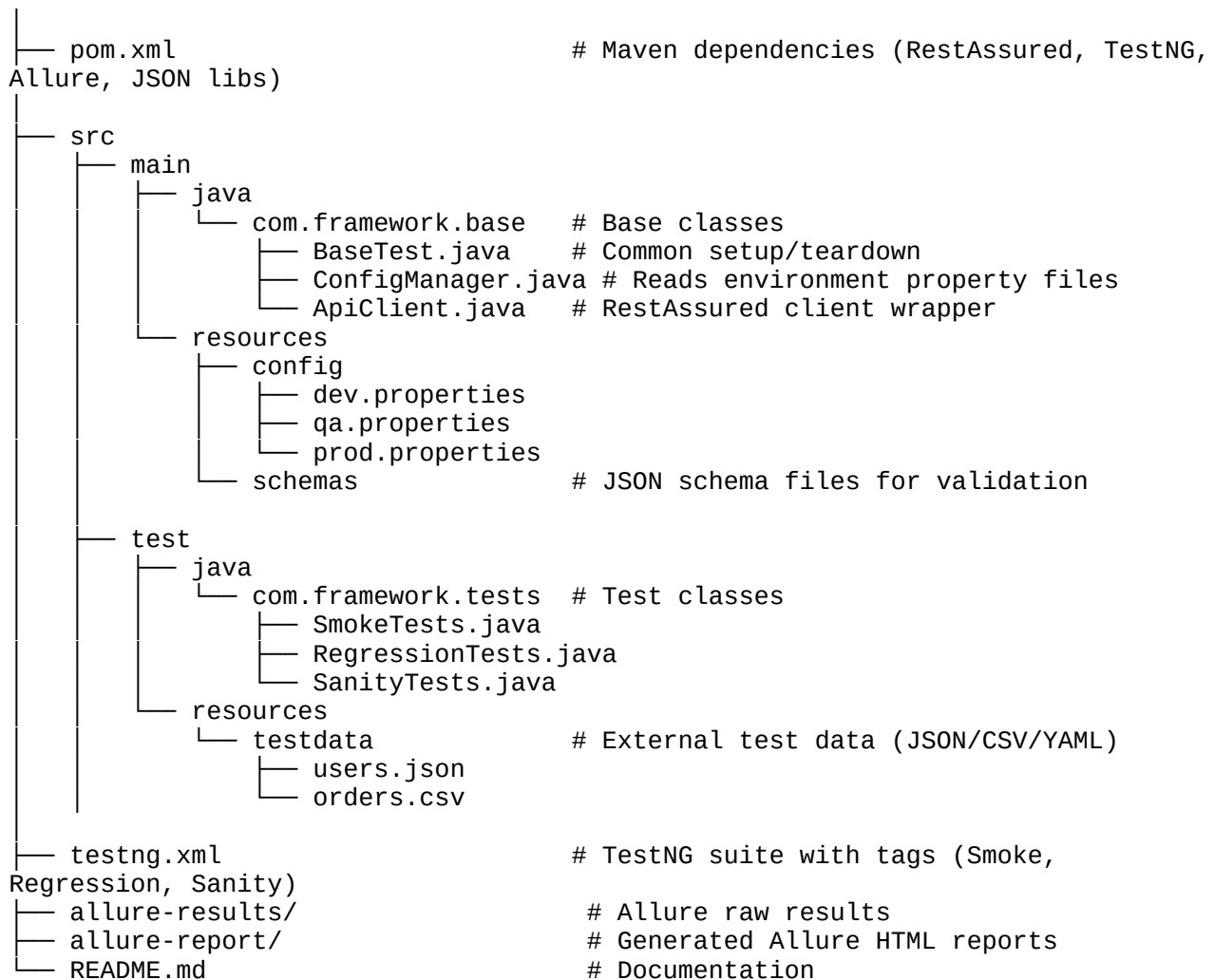
Great, Srinivasan — you've selected **C (GitLab CI/CD)** for integration. That gives us a strong DevOps pipeline option with flexibility for version control and automation.

✅ Now we've completed all **7 questions**. Based on your choices, here's the **initial framework structure** for the RestAssured + TestNG automation framework:

📁 Framework Structure (Maven + TestNG + RestAssured + Allure)

Code

RestAssured-TestNG-Framework/



Key Highlights Based on Your Choices

- **Maven** → Dependency management and CI/CD integration.
- **Allure Reports** → Interactive reporting.
- **External Test Data (JSON/CSV/YAML)** → Flexible test input.
- **Property Files per Environment** → Clean environment handling.
- **Validation (Status + Body + Schema)** → Robust API checks.
- **Tag-based Execution (Smoke/Regression/Sanity)** → Flexible suite runs.
- **GitLab CI/CD** → Pipeline integration for automation.

 Next Step: I'll start with **Step 1: Maven pom.xml setup** (dependencies for RestAssured, TestNG, Allure, JSON handling). Would you like me to proceed with **Step 1 implementation** now?

Proceed

Alright Srinivasan, let's move into **Step 1: Maven pom.xml setup** for your RestAssured + TestNG + Allure framework. Since you selected Maven, Allure Reports, external test data, property files for environments, full validation, tag-based execution, and GitLab CI/CD, here's the dependency setup:

Step 1: pom.xml Dependencies

xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.framework</groupId>
  <artifactId>restassured-testng-framework</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <!-- TestNG -->
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.9.0</version>
      <scope>test</scope>
    </dependency>

    <!-- RestAssured -->
    <dependency>
      <groupId>io.rest-assured</groupId>
      <artifactId>rest-assured</artifactId>
      <version>5.4.0</version>
      <scope>test</scope>
    </dependency>

    <!-- JSON Schema Validator -->
    <dependency>
      <groupId>io.rest-assured</groupId>
      <artifactId>json-schema-validator</artifactId>
      <version>5.4.0</version>
      <scope>test</scope>
    </dependency>

    <!-- Jackson (for JSON parsing) -->
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.17.0</version>
    </dependency>

    <!-- Allure TestNG Integration -->
    <dependency>
      <groupId>io.qameta.allure</groupId>
      <artifactId>allure-testng</artifactId>
      <version>2.24.0</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- Maven Surefire Plugin for TestNG -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
        <configuration>
          <suiteXmlFiles>
```

```

        <suiteXmlFile>testng.xml</suiteXmlFile>
    </suiteXmlFiles>
</configuration>
</plugin>

<!-- Allure Maven Plugin -->
<plugin>
    <groupId>io.qameta.allure</groupId>
    <artifactId>allure-maven</artifactId>
    <version>2.11.2</version>
</plugin>
</plugins>
</build>
</project>

```



What this does:

- **TestNG** → Test execution engine.
- **RestAssured** → API automation library.
- **JSON Schema Validator** → Schema validation support.
- **Jackson** → JSON parsing for external test data.
- **Allure TestNG** → Reporting integration.
- **Surefire Plugin** → Runs TestNG suites via Maven.
- **Allure Maven Plugin** → Generates Allure reports.



Srinivasan, please review and confirm if this **Step 1 (pom.xml setup)** looks good. Once approved, I'll proceed to **Step 2: Framework Base Classes (BaseTest, ConfigManager, ApiClient)**.

Proceed

Alright Srinivasan, let's move into **Step 2: Framework Base Classes**. These classes will form the backbone of your RestAssured + TestNG automation framework.



Step 2: Base Classes

1. BaseTest.java

Handles common setup/teardown for TestNG tests, including RestAssured configuration and Allure integration.

java

```

package com.framework.base;

import io.restassured.RestAssured;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.AfterClass;

public class BaseTest {

    @BeforeClass
    public void setUp() {
        // Load environment configuration
    }
}

```

```

        String baseUrl = ConfigManager.getProperty("base.uri");
        RestAssured.baseURI = baseUrl;
    }

    @AfterClass
    public void tearDown() {
        // Any cleanup if required
    }
}

```

2. ConfigManager.java

Reads environment-specific property files (Dev, QA, Prod).

```

java

package com.framework.base;

import java.io.FileInputStream;
import java.io.IOException;
import java.util.Properties;

public class ConfigManager {
    private static Properties properties = new Properties();

    public static void load(String env) {
        try {
            FileInputStream fis = new
FileInputStream("src/main/resources/config/" + env + ".properties");
            properties.load(fis);
        } catch (IOException e) {
            throw new RuntimeException("Failed to load config file for
environment: " + env, e);
        }
    }

    public static String getProperty(String key) {
        return properties.getProperty(key);
    }
}

```

3. ApiClient.java

Wrapper for RestAssured requests to keep tests clean and reusable.

```

java

package com.framework.base;

import io.restassured.response.Response;
import static io.restassured.RestAssured.given;

public class ApiClient {

    public static Response get(String endpoint) {
        return given()
            .when()
            .get(endpoint)
            .then()
            .extract()
            .response();
    }
}

```

```

    public static Response post(String endpoint, Object body) {
        return given()
            .header("Content-Type", "application/json")
            .body(body)
            .when()
            .post(endpoint)
            .then()
            .extract()
            .response();
    }

    public static Response put(String endpoint, Object body) {
        return given()
            .header("Content-Type", "application/json")
            .body(body)
            .when()
            .put(endpoint)
            .then()
            .extract()
            .response();
    }

    public static Response delete(String endpoint) {
        return given()
            .when()
            .delete(endpoint)
            .then()
            .extract()
            .response();
    }
}

```



What we achieved in Step 2:

- **BaseTest** → Common setup/teardown.
- **ConfigManager** → Environment property file handling.
- **ApiClient** → Centralized RestAssured request methods.



Srinivasan, please review and confirm if this **Step 2 (Base Classes)** looks good. Once approved, I'll proceed to **Step 3: Test Data Management (JSON/CSV/YAML handling utilities)**.

Proceed

Alright Srinivasan, let's move into **Step 3: Test Data Management Utilities**. Since you selected **external files (JSON/CSV/YAML)**, we'll build utility classes to handle them cleanly.



Step 3: Test Data Utilities

1. JsonUtils.java

Utility to read JSON test data files.

java

```

package com.framework.utils;

import com.fasterxml.jackson.databind.ObjectMapper;
import java.io.File;

```

```

import java.io.IOException;

public class JsonUtils {

    private static final ObjectMapper mapper = new ObjectMapper();

    public static <T> T readJson(String filePath, Class<T> clazz) {
        try {
            return mapper.readValue(new File(filePath), clazz);
        } catch (IOException e) {
            throw new RuntimeException("Failed to read JSON file: " + filePath,
e);
        }
    }
}

```

2. CsvUtils.java

Utility to read CSV test data files.

```

java

package com.framework.utils;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class CsvUtils {

    public static List<String[]> readCsv(String filePath) {
        List<String[]> records = new ArrayList<>();
        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            String line;
            while ((line = br.readLine()) != null) {
                String[] values = line.split(",");
                records.add(values);
            }
        } catch (IOException e) {
            throw new RuntimeException("Failed to read CSV file: " + filePath,
e);
        }
        return records;
    }
}

```

3. YamlUtils.java

Utility to read YAML test data files.

```

java

package com.framework.utils;

import org.yaml.snakeyaml.Yaml;
import java.io.InputStream;
import java.util.Map;

public class YamlUtils {

```

```

    public static Map<String, Object> readYaml(String filePath) {
        Yaml yaml = new Yaml();
        try (InputStream inputStream =
YamlUtils.class.getClassLoader().getResourceAsStream(filePath)) {
            return yaml.load(inputStream);
        } catch (Exception e) {
            throw new RuntimeException("Failed to read YAML file: " + filePath,
e);
        }
    }
}

```

What we achieved in Step 3:

- **JsonUtils** → Reads structured JSON test data.
- **CsvUtils** → Reads tabular CSV test data.
- **YamlUtils** → Reads YAML configs/test data.
- All utilities are reusable and keep test classes clean.

 Srinivasan, please review and confirm if this **Step 3 (Test Data Utilities)** looks good. Once approved, I'll proceed to **Step 4: Environment Property Files (Dev, QA, Prod setup)**.

Proceed

Alright Srinivasan, let's move into **Step 4: Environment Property Files**. Since you selected **property files per environment (Dev, QA, Prod)**, we'll create simple `.properties` files to manage environment-specific configurations.

Step 4: Environment Property Files

1. dev.properties

properties

```

base.uri=https://dev-api.company.com
auth.token=dev-token-123
timeout=5000

```

2. qa.properties

properties

```

base.uri=https://qa-api.company.com
auth.token=qa-token-456
timeout=5000

```

3. prod.properties

properties

```

base.uri=https://prod-api.company.com
auth.token=prod-token-789
timeout=10000

```

How this works:

- Each environment has its own **base URI**, **auth token**, and **timeout** values.
- The `ConfigManager` (from Step 2) will load the correct file based on the environment parameter passed (e.g., `ConfigManager . load( "qa" )`).
- This keeps your framework flexible and avoids hardcoding environment details in test classes.

 Srinivasan, please review and confirm if this **Step 4 (Environment Property Files)** looks good. Once approved, I'll proceed to **Step 5: TestNG Suite Setup (Smoke, Regression, Sanity with tags)**.

Proceed

Alright Srinivasan, let's move into **Step 5: TestNG Suite Setup**. Since you selected **tag-based execution (Smoke, Regression, Sanity)**, we'll configure `testng.xml` to allow flexible runs.

Step 5: TestNG Suite Setup

`testng.xml`

xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd" >
<suite name="API Automation Suite" verbose="1" parallel="false">

    <!-- Smoke Suite -->
    <test name="Smoke Tests">
        <packages>
            <package name="com.framework.tests">
                <classes>
                    <class name="com.framework.tests.SmokeTests"/>
                </classes>
            </package>
        </packages>
    </test>

    <!-- Regression Suite -->
    <test name="Regression Tests">
        <packages>
            <package name="com.framework.tests">
                <classes>
                    <class name="com.framework.tests.RegressionTests"/>
                </classes>
            </package>
        </packages>
    </test>

    <!-- Sanity Suite -->
    <test name="Sanity Tests">
        <packages>
            <package name="com.framework.tests">
                <classes>
                    <class name="com.framework.tests.SanityTests"/>
                </classes>
            </package>
        </packages>
    </test>
</suite>
```

</suite>

How this works:

- **SmokeTests.java** → Quick validation of critical endpoints.
- **RegressionTests.java** → Full coverage of APIs.
- **SanityTests.java** → Lightweight checks for environment stability.
- You can run specific suites by passing `-DsuiteXmlFile=testng.xml` with Maven or configure GitLab CI/CD to trigger specific tags.

 Srinivasan, please review and confirm if this **Step 5 (TestNG Suite Setup)** looks good. Once approved, I'll proceed to **Step 6: Sample Test Class Implementation (with RestAssured + Allure + Schema validation)**.

Proceed

Alright Srinivasan, let's move into **Step 6: Sample Test Class Implementation**. This will demonstrate how your framework ties together **RestAssured + TestNG + Allure + Schema validation** with external test data.

Step 6: Sample Test Class

SmokeTests.java

java

```
package com.framework.tests;

import com.framework.base.ApiClient;
import com.framework.base.BaseTest;
import io.qameta.allure.Description;
import io.qameta.allure.Step;
import io.restassured.response.Response;
import org.testng.Assert;
import org.testng.annotations.Test;

import static
io.restassured.module.jsv.JsonSchemaValidator.matchesJsonSchemaInClasspath;

public class SmokeTests extends BaseTest {

    @Test(priority = 1, groups = {"smoke"})
    @Description("Verify GET Users API returns 200 and valid schema")
    public void testGetUsers() {
        Response response = ApiClient.get("/users");

        logResponse(response);

        // Status code validation
        Assert.assertEquals(response.getStatusCode(), 200, "Status code
mismatch");

        // Body field validation
        Assert.assertTrue(response.jsonPath().getList("id").size() > 0, "No
users found");
    }
}
```

```

        // Schema validation
        response.then().assertThat()
            .body(matchesJsonSchemaInClasspath("schemas/users-
schema.json"));
    }

    @Step("Logging API response")
    private void logResponse(Response response) {
        System.out.println("Response: " + response.asString());
    }
}

```

Key Highlights:

- **@Test(groups = {"smoke"})** → Allows tag-based execution (Smoke suite).
- **Allure @Description & @Step** → Adds rich reporting context.
- **ApiClient wrapper** → Keeps test clean and reusable.
- **Schema validation** → Ensures response structure matches expected JSON schema.
- **External schema file** → Located in `src/main/resources/schemas/users-schema.json`.

 Srinivasan, please review and confirm if this **Step 6 (Sample Test Class Implementation)** looks good. Once approved, I'll proceed to **Step 7: GitLab CI/CD Integration (pipeline configuration for Maven + TestNG + Allure)**.

Proceed

Alright Srinivasan, let's move into **Step 7: GitLab CI/CD Integration**. Since you selected **GitLab CI/CD**, we'll configure a pipeline to run your RestAssured + TestNG + Allure framework automatically.

Step 7: GitLab CI/CD Setup

.gitlab-ci.yml

yaml

```

stages:
  - test
  - report

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=.m2/repository"

cache:
  paths:
    - .m2/repository

test_api:
  stage: test
  image: maven:3.9.6-eclipse-temurin-17
  script:
    - mvn clean test -DsuiteXmlFile=testng.xml
  artifacts:
    paths:
      - target/surefire-reports

```

```
    - allure-results
    expire_in: 1 week

generate_report:
  stage: report
  image: maven:3.9.6-eclipse-temurin-17
  script:
    - mvn allure:report
  artifacts:
    paths:
      - allure-report
    expire_in: 1 week
```

Key Highlights:

- **Stages** → test (runs API tests) and report (generates Allure HTML report).
- **Maven image** → Uses JDK 17 + Maven for builds.
- **Artifacts** → Stores surefire-reports, allure-results, and final allure-report for download.
- **Cache** → Speeds up builds by caching Maven dependencies.
- **Triggering suites** → You can run specific suites by passing variables like -DsuiteXmlFile=testng.xml or -Dgroups=smoke.

 With this, your **framework design is complete:**

1. **Maven setup**
2. **Base classes**
3. **Test data utilities**
4. **Environment property files**
5. **TestNG suite setup**
6. **Sample test class**
7. **GitLab CI/CD pipeline**